

Übungsblatt 8: KI-Beschleuniger & Edge AI

Mikrocomputertechnik 3 (MCT3)

Musterlösung

Zielsetzung

Sie vertiefen Vorlesung 8: Memory Wall bei neuronalen Netzen, Data-Level Parallelism / RVV, Systolic Arrays (NPUs) sowie Edge-AI-Praxis mit TensorFlow Lite Micro (Quantisierung, Arena, Inferenz).

Bearbeitungszeit: ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 12 P, Aufgabe 2: 12 P (Gesamt: 24 P)

Teil 1: Theorie — Von der CPU zum KI-Beschleuniger

Aufgabe 1.1: Die Memory Wall bei neuronalen Netzen

Neuronale Netze bestehen hardwareseitig primär aus großen Matrizen. Die grundlegendste Operation ist die MAC-Operation (Multiply-Accumulate).

1. Erläutern Sie die Struktur $a \leftarrow a + (w \times x)$ und die Bedeutung der Variablen.
2. Warum „verhungert“ eine klassische OoO-CPU an der *Memory Wall*? Argumentieren Sie mit der *Arithmetic Intensity* (Rechnen vs. Speicherzugriffe).
3. Warum helfen Branch Prediction und Reservation Stations bei Matrix-Multiplikationen kaum und verschwenden oft Energie?

Musterlösung Aufgabe 1.1

1. w = Gewicht, x = Eingabe/Activation, a = Akkumulator (laufende Summe). Eine MAC multipliziert und addiert in einem Schritt.
2. Pro MAC typisch zwei Loads (w, x) für eine Mul+Add $\rightarrow$ geringe Arithmetic Intensity. Die ALU ist schnell fertig, wartet dann oft viele Takte auf DRAM/Cache $\rightarrow$ Memory Wall, geringe ALU-Auslastung.
3. Matrix-Schleifen sind linear und hoch vorhersagbar: der Predictor rät ohnehin meist richtig (wenig Mehrwert). Reservation Stations / OoO verbrauchen Energie, um Abhängigkeiten zu managen — obwohl viele MACs voneinander unabhängig sind und eher DLP brauchen.

Aufgabe 1.2: Vektoren und die RISC-V Vector Extension (RVV)

1. Beschreiben Sie *Data-Level Parallelism* (DLP) im Kontrast zu ILP. Warum spart DLP Energie in Fetch/Decode?
2. RVV gilt als hardware-agnostisch. Wie ermöglicht `vsetvli`, dass derselbe Maschinencode auf schmaler Edge-CPU und breitem Server-Node läuft, ohne Neu-Kompilierung?

Musterlösung Aufgabe 1.2

1. **ILP:** mehrere *verschiedene* Instruktionen parallel. **DLP:** *eine* Instruktion (z. B. Vector-Add) auf viele Datenelemente. Fetch/Decode läuft einmal pro Vektorbefehl statt pro Skalar-Addition → weniger Front-End-Energie.
2. `vsetvli` fragt zur Laufzeit die effektive Vektorlänge (VLEN) ab; die Schleife passt die Chunk-Größe an. Schmale Hardware braucht mehr Iterationen, breite weniger — gleicher Code.

Aufgabe 1.3: Systolic Arrays (NPU's)

1. Erklären Sie den Aufbau eines Systolic Arrays. Wie durchbricht es den Von-Neumann-Flaschenhals bezüglich RAM-Zugriffen?
2. Was ist *Weight-Stationary* Dataflow, und warum spart das Energie?
3. Nennen Sie einen entscheidenden Nachteil gegenüber einer General-Purpose-CPU.

Musterlösung Aufgabe 1.3

1. 2D-Gitter vieler einfacher MAC-Zellen; oft nur Randzellen am Speicherbus. Daten werden taktsynchron von Zelle zu Zelle weitergereicht (Eimerkette). Ein DRAM-Load speist viele MACs → Wiederverwendung, weniger Bus-Last.
2. Gewichte werden einmal geladen und in lokalen Registern der Zellen gehalten; nur Activations fließen weiter. Weniger Datenbewegung auf dem Die → Energieeinsparung.
3. Extrem domänenspezifisch / starr: kaum klassischer Kontrollfluss — ungeeignet für OS, Netzwerkstack oder allgemeine C-Logik.

Teil 2: Praxis — Edge AI mit TensorFlow Lite Micro

Szenario: Ein trainiertes Modell (Schrittklassifikation) soll auf einem RISC-V-MCU laufen. Eingabe: 3-Achsen-Beschleunigung (Anknüpfung an I2C aus VL5).

Aufgabe 2.1: Das Modell im Flash-Speicher

1. Warum quantisiert man Modelle für MCUs (Float32 $\rightarrow$ Int8)? Nennen Sie zwei Vorteile.
2. Das `.tf-lite`-Modell liegt als C-Byte-Array vor. Mit welchem Keyword und welchem Linker-Verhalten landet es im Flash statt im SRAM?

Musterlösung Aufgabe 2.1

1. **Footprint:** ca. $4\times$ kleiner (4 Byte $\rightarrow$ 1 Byte). **Performance/Energie:** Integer-ALU oft schneller/effizienter als Float-Emulation ohne FPU.
2. Keyword `const` $\rightarrow$ typisch `.rodata`; das Linker-Skript mappt Read-Only-Daten in den NOR-Flash (XIP/ROM), ohne das Array beim Start in den SRAM zu kopieren.

Aufgabe 2.1b: Modellgröße Float32 vs. Int8

Ein kleines Netz habe 250 000 Gewichte (Biases vernachlässigt).

1. Berechnen Sie den Speicherbedarf der Gewichte in Float32 (4 Byte/Gewicht) und in Int8 (1 Byte/Gewicht).
2. Um welchen Faktor schrumpft der Gewichtsspeicher durch Int8-Quantisierung?

Musterlösung Aufgabe 2.1b

1. Float32: $250\,000 \times 4\text{ B} = 1\,000\,000\text{ B} = 1\text{ MB}$.
Int8: $250\,000 \times 1\text{ B} = 250\,000\text{ B} = 250\text{ KB}$.
2. Faktor 4 (Float32 ist viermal so groß) bzw. Int8 benötigt $1/4$ des Float32-Speichers.

Aufgabe 2.2: TFLM-Inferenzzyklus (C++)

Vervollständigen Sie das Snippet. Hilfsobjekte `resolver`, `error_reporter`, `input`, `output` und `read_i2c_sensor()` seien gegeben.

Hinweis: Das Snippet nutzt Float-Tensoren (`data.f`). Bei einem Int8-quantisierten Modell greifen Sie typisch auf `data.int8` zu und skalieren mit den Quantisierungsparametern (`scale`, `zero_point`) — hier didaktisch als Float belassen.

```
#include "tensorflow/lite/micro/all_ops_resolver.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
/* Modell-Array step_model_tflite ist inkludiert */

/* 1. Tensor Arena: 10 KiB, global */
/* TODO */

void setup_and_run_ai() {
    const tflite::Model *model = tflite::GetModel(step_model_tflite);

    tflite::MicroInterpreter interpreter(
        model, resolver, tensor_arena, 10 * 1024, error_reporter);

    /* 2. Arena vorbereiten */
    /* TODO: interpreter.... */

    while (1) {
        /* 3. Input: 128 Werte via read_i2c_sensor() */
        /* TODO */

        /* 4. Inferenz starten */
        /* TODO: interpreter.... */

        float score_laufen = output->data.f[2];
        /* 5. bei Wahrscheinlichkeit > 80 % melden */
        /* TODO: if (...) { printf(...); } */
    }
}
```

Musterlösung Aufgabe 2.2

```
#include "tensorflow/lite/micro/all_ops_resolver.h"
#include "tensorflow/lite/micro/micro_interpreter.h"

const int kTensorArenaSize = 10 * 1024;
uint8_t tensor_arena[kTensorArenaSize];

void setup_and_run_ai() {
    const tflite::Model *model = tflite::GetModel(step_model_tflite);

    tflite::MicroInterpreter interpreter(
        model, resolver, tensor_arena, kTensorArenaSize, error_reporter);

    interpreter.AllocateTensors();
}
```

```

while (1) {
    for (int i = 0; i < 128; i++)
        input->data.f[i] = read_i2c_sensor();

    interpreter.Invoke();

    float score_laufen = output->data.f[2];
    if (score_laufen > 0.8f)
        printf("Nutzer laeuft!\n");
}
}

```

(Hinweis: In realem TFLM-Code holt man `input/output` typisch über `interpreter.input(0)` / `output(0)` nach `AllocateTensors()` — hier didaktisch vereinfacht. Bei Int8-Modellen: `data.int8` statt `data.f`, plus De-/Quantisierung.)

Aufgabe 2.3: System-Auswirkungen in Renode

Sie führen das Programm in Renode auf einem RISC-V-SoC *ohne* NPU/RVV aus.

1. Was beobachten Sie bezüglich der Ausführungszeit von `Invoke()`?
2. Warum unterstreicht das die Notwendigkeit domänenspezifischer Architekturen am Edge?

Musterlösung Aufgabe 2.3

1. `Invoke()` blockiert die CPU für sehr viele Takte (hunderttausende bis Millionen) — die Allzweck-CPU rechnet MACs mühsam über `lw/mul/sw`; die Memory Wall wird spürbar (träge Simulation/Terminal).
2. Währenddessen fehlen Zyklen für System-Ticks, Busse, Sensorik. Für anspruchsvolle Edge-KI braucht es daher oft einen Beschleuniger (NPU/Vektor) als Co-Prozessor.